Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Bases de Datos II

Proyecto TecDigitalito

Grupo 2

Prof. Erick Fabian Hernández Bonilla

Estudiantes:

Paulo César Herrera Arias - 2023107673

Dayron Josué Porras Perez - 2023124693

Ian Sebastián Ugalde Muñoz - 2024124076

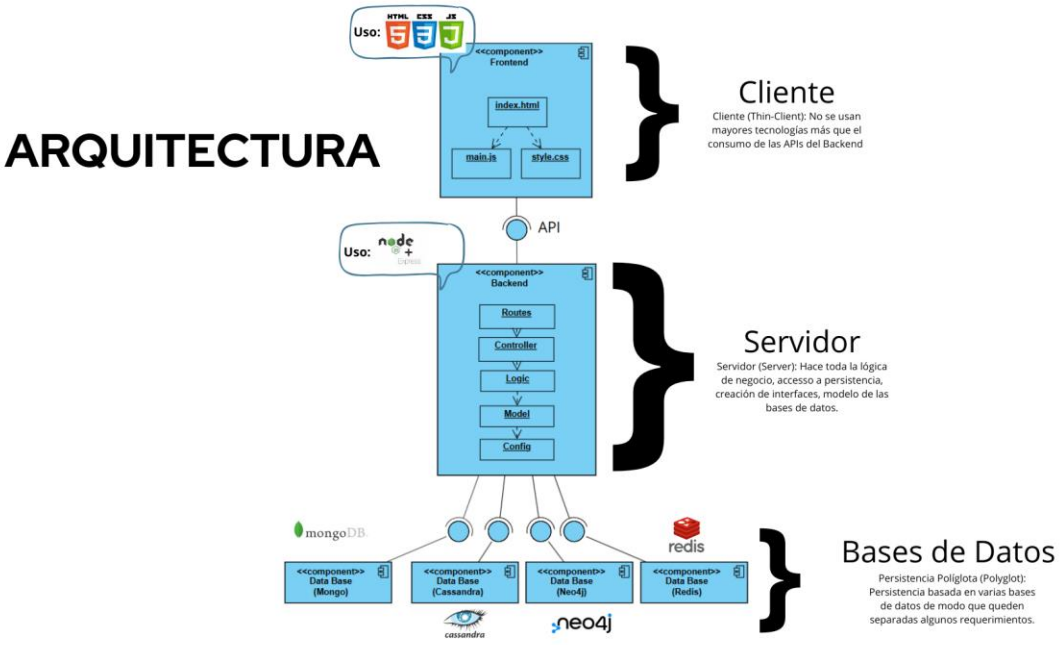
Josué Santiago Hidalgo Sandoval - 2024800128

Abril, 2026

Introducción

TEC Digitalito es una plataforma de clases virtuales. El sistema permite a los docentes crear y publicar cursos con contenido estructurado y evaluaciones, mientras que los estudiantes pueden matricularse, consumir el contenido y rendir dichas evaluaciones. Adicionalmente, la plataforma incorpora funcionalidades sociales (como gestión de amigos y mensajería directa) y un sistema de auditoría.

Arquitectura



Fuente: Elaboración Propia

Conexiones

API:

Node.js es un entorno de ejecución de JavaScript del lado del servidor que permite construir aplicaciones escalables y de alto rendimiento (NodeJS, s. f.). Las API no son más que interfaces creadas con ayuda de Express el cual actúa como una capa delgada sobre el servidor HTTP de Node para simplificar la creación de aplicaciones web, APIs y servicios, gestionando rutas y solicitudes HTTP de manera eficiente sin imponer una estructura rígida.

Una vez creadas las APIs el Cliente (Frontend) consume las APIs para realizar ciertas operaciones que no le interesa su proceso más que el resultado, el cliente como se explica en la imagen no hace ninguna funcionalidad más que mostrar al Usuario.

Conexiones a Bases de Datos (Config):

Las conexiones con las Bases de Datos se pueden resumir en el paquete de config dentro del Backend. En ella se detallan las URIs que necesita la base de datos para poder ser accesada. En algunas ocaciones necesita que se le especifique el modelo en caso de ser la primer conección por lo que se trae el modelo del paquete de models para crear el modelo y que funcione después para su uso.

Tecnologías

Frontend

- HTML
- CSS
- JS

Backend

- NodeJS
- ExpressJS

Bases de Datos

- MongoDB
- Redis
- Neo4j
- Cassandra

Diseño de Bases de Datos ¿Por qué el uso de las Bases de Datos?

El sistema utiliza cuatro bases de datos, cada una seleccionada según sus fortalezas técnicas y el patrón de acceso de los datos que gestiona. A continuación, se justifica cada elección y se describe qué información se almacena y por qué.

MongoDB

MongoDB se utiliza como la base de datos principal del sistema, encargada de almacenar la mayor parte de la información de negocio.

Su elección se fundamenta en las siguientes razones:

- **Flexibilidad de esquema**
  Permite manejar estructuras heterogéneas y campos opcionales sin necesidad de definir un esquema rígido, lo cual simplifica la evolución del modelo de datos (MongoDB, s. f.).
- **Modelado natural de jerarquías**
  La estructura basada en documentos facilita representar relaciones como curso → sección → contenido de forma intuitiva, evitando la complejidad de múltiples JOINs.
- **Soporte de índices eficientes**
  Incluye índices de texto completo que permiten implementar búsquedas sobre cursos de manera eficiente en una sola consulta (MongoDB, s. f.).
- **Atomicidad a nivel de documento**
  Garantiza consistencia en operaciones que afectan una sola entidad, como actualizaciones de estado o cambios de información del usuario (MongoDB, s. f.).

En conjunto, MongoDB actúa como la **fuente de verdad** del sistema. La integración con Node.js se realiza de forma directa mediante la conexión oficial, lo que facilita la comunicación y operación con la base de datos desde el backend (Schaefer, 2026).

Redis

Redis se emplea como una **capa de almacenamiento en memoria** orientada a datos temporales y de alta frecuencia de acceso. Redis actúa como una capa de alta velocidad para datos efímeros, permitiendo accesos en memoria con latencia muy baja y soporte nativo para expiración automática mediante TTL. Además, proporciona estructuras de datos especializadas como listas y conjuntos que optimizan operaciones frecuentes del sistema (Redis, 2026).

Las principales razones de su uso son:

- **Alto rendimiento**
  Permite accesos en tiempo constante con latencias muy bajas, ideal para validación de sesiones en cada solicitud (Redis, 2026).
- **Expiración automática (TTL)**
  Facilita la gestión de datos efímeros como sesiones, tokens y bloqueos sin necesidad de procesos adicionales de limpieza (Redis, 2026).
- **Estructuras de datos especializadas**
  Como mencionaba Hebda (2025) en su reporte, redis soporta tipos como listas y conjuntos, útiles para manejar colas de notificaciones o agrupaciones de sesiones por usuario.
- **Control de seguridad**
  Permite implementar mecanismos como limitación de intentos de login y bloqueos temporales de cuentas (Redis, 2026).

Redis funciona como un **componente auxiliar de alto rendimiento**.

Neo4j

Neo4j se utiliza para gestionar las **relaciones entre usuarios y cursos**, especialmente en el contexto social.

Su uso se justifica por:

- **Modelo orientado a grafos**
  Permite representar de forma directa relaciones como amistades o matriculaciones (Neo4j, s. f.).
- **Consultas eficientes de relaciones**
  Operaciones como “amigos en común” o “usuarios en el mismo curso” se resuelven mediante recorridos de grafo, evitando consultas complejas (Neo4j, s. f.).
- **Escalabilidad en redes de relaciones**
  A medida que crece el número de usuarios, las consultas siguen siendo eficientes (Neo4j, s. f.).
- **Optimización de consultas sociales**
  Reduce significativamente la complejidad en comparación con bases de datos relacionales (Neo4j, s. f.).

Neo4j permite manejar de forma eficiente la **lógica social del sistema**.

Cassandra

Apache Cassandra se emplea para el almacenamiento de **logs y auditoría del sistema**.

Las razones principales son:

- **Optimización para escritura intensiva**
  Está diseñada para manejar grandes volúmenes de inserciones con baja latencia (Apache Cassandra, s. f.).
- **Alta disponibilidad**
  Su arquitectura distribuida garantiza que el sistema continúe funcionando incluso ante fallos (Apache Cassandra, s. f.).
- **Modelo orientado a consultas**
  Las tablas se diseñan específicamente según cómo se consultarán los datos, lo que mejora el rendimiento (Apache Cassandra, s. f.).
- **Consistencia eventual adecuada para logs**
  Dado que los registros de auditoría no requieren consistencia inmediata, Cassandra resulta una opción eficiente (Apache Cassandra, s. f.).
- **Ordenamiento eficiente por tiempo**
  Permite obtener los eventos más recientes sin procesamiento adicional (Apache Cassandra, s. f.).

Cassandra se utiliza como un **sistema especializado de auditoría**.

Modelado de Datos por Base de Datos

MongoDB

Colección: users

- _id: user_id
- {

```
    "_id": "uuid",
    "username": "string",
    "password": "string (bcrypt hash)",
    "salt": "string",
    "nombre_completo": "string",
    "fecha_nacimiento": "date",
    "avatar": "string (url o base64)",
    "email": "string",
    "password_history": [
        {
            "password": "string (hash anterior)",
            "changed_at": "date"
        }
    ],
    "created_at": "date",
    "updated_at": "date"
}
```

Colección: courses

- _id: course_id (UUID)
- Índice: docente.user_id, publicado
- Índice compuesto: (publicado, nombre) — para búsqueda de cursos activos

```
{
    "_id": "uuid",
    "codigo": "string",
    "nombre": "string",
    "descripcion": "string",
    "fecha_inicio": "date",
    "fecha_fin": "date | null",
```

```
    "foto": "string (url)",
    "publicado": false,
    "estado": "borrador | activo | terminado",
    "docente": {
        "user_id": "ref → users._id",
        "nombre": "string (desnormalizado para evitar join)"
    },
    "original_course_id": "ref → courses._id | null",
    "created_at": "date",
    "updated_at": "date"
}
```

Colección: sections

- _id: section_id (UUID)
- Índice: course_id
- Índice compuesto: (course_id, parent_section_id) — para construir el árbol por nivel

```
{
    "_id": "uuid",
    "course_id": "ref → courses._id",
    "titulo": "string",
    "descripcion": "string | null",
    "parent_section_id": "ref → sections._id | null",
    "orden": 1,
    "created_at": "date"
}
```

Colección: contents

- _id: content_id (UUID)
- Índice: section_id

```
{
    "_id": "uuid",
    "section_id": "ref → sections._id",
    "tipo": "texto | documento | video | imagen",
    "orden": 1,
```

```
    "data": {
        "texto": "string | null",
        "url": "string | null",
        "nombre_archivo": "string | null",
        "duracion_segundos": "number | null",
        "mime_type": "string | null"
    },
    "created_at": "date"
}
```

Colección: evaluations

- _id: evaluation_id (UUID)
- Índice: course_id
- Índice compuesto: (course_id, fecha_inicio, fecha_fin) — para filtrar evaluaciones activas

```
{
    "_id": "uuid",
    "course_id": "ref → courses._id",
    "titulo": "string",
    "descripcion": "string | null",
    "fecha_inicio": "date",
    "fecha_fin": "date",
    "visible_para_estudiante": true,
    "preguntas": [
        {
            "question_id": "uuid",
            "enunciado": "string",
            "orden": 1,
            "opciones": [
                {
                    "option_id": "uuid",
                    "texto": "string",
                    "es_correcta": false
                }
            ]
        }
    ],
    "created_at": "date",
    "updated_at": "date"
}
```

Colección: enrollments

- _id: enrollment_id (UUID)
- Índice compuesto único: (user_id, course_id) — previene matrículas duplicadas

- Índice: course_id — para listar estudiantes de un curso (HU-16)
- {
    "_id": "uuid",
    "user_id": "ref → users._id",
    "course_id": "ref → courses._id",
    "fecha": "date",
    "estado": "activo | completado | abandonado"
  }

Colección: submissions

- _id: submission_id (UUID)
- Índice: (user_id, course_id) — para HU-25: todas las evaluaciones del estudiante en un curso
- Índice: (user_id, evaluation_id) — unicidad por intento (si solo se permite un intento)
- {

```
    "_id": "uuid",
    "user_id": "ref → users._id",
    "evaluation_id": "ref → evaluations._id",
    "course_id": "ref → courses._id",
    "fecha": "date",
    "calificacion": 85.0,
    "total_preguntas": 10,
    "correctas": 7,
    "respuestas": [
        {
            "question_id": "ref →
evaluations.preguntas.question_id",
            "option_id": "ref →
evaluations.preguntas.opciones.option_id",
            "es_correcta": true
        }
    ]
}
```

Colección: course_messages

- _id: message_id (UUID)
- Índice: course_id — para listar todas las consultas de un curso (vista docente)
- Índice compuesto: (course_id, sender_id) — para filtrar mensajes por estudiante

```
{
    "_id": "uuid",
    "course_id": "ref → courses._id",
    "sender_id": "ref → users._id",
```

```
    "sender_nombre": "string (desnormalizado)",
    "tipo": "consulta | respuesta",
    "contenido": "string",
    "fecha": "date",
    "respuesta_a": "ref → course_messages._id | null",
    "leido": false
}
```

Colección: direct_messages

- _id: message_id (UUID)
- Índice compuesto: (sender_id, receiver_id) — conversación entre dos usuarios
- Índice: receiver_id — para bandeja de entrada del receptor

```
{
    "_id": "uuid",
    "sender_id": "ref → users._id",
    "receiver_id": "ref → users._id",
    "sender_nombre": "string (desnormalizado)",
    "contenido": "string",
    "fecha": "date",
    "respuesta_a": "ref → direct_messages._id | null",
    "leido": false
}
```

Redis

Bucket: sessions

- Key: `session:{session_id}`
- TTL: configurable por política (recomendado: 2 horas de inactividad, sliding window)

```
{
    "user_id": "ref → users._id",
    "token": "string (JWT o token opaco)",
    "ip": "string",
    "dispositivo": "string",
    "user_agent": "string",
    "created_at": "timestamp",
    "expires_at": "timestamp"
}
```

Bucket: remember_me

- Key: `remember:{user_id}`
- TTL: configurable (recomendado: 30 días)

```
{
    "token": "string (token aleatorio seguro, mínimo 32 bytes)",
    "ip_origen": "string",
    "user_agent_origen": "string",
    "created_at": "timestamp",
    "expires_at": "timestamp"
}
```

Bucket: login_attempts

- Key: `attempts:{user_id}`
- TTL: ventana deslizante (recomendado: 15 minutos desde el último intento fallido)

```
{
    "attempts": 0,
    "last_attempt": "timestamp",
    "ip_ultimo": "string"
}
```

Bucket: user_blocks

- Key: `block:{user_id}`
- TTL: igual al tiempo de bloqueo (recomendado: 15–30 minutos); Redis expira la key automáticamente

```
{
    "blocked": true,
    "reason": "max_intentos_fallidos | sospecha | manual",
    "blocked_at": "timestamp",
    "expires_at": "timestamp",
    "ip_trigger": "string"
}
```

Bucket: course_messages

- Key: `reset:{token}`
- TTL: igual a `expires_at` (recomendado: 15–30 minutos)

```
{
    "user_id": "ref → users._id",
    "email": "string",
    "created_at": "timestamp",
    "expires_at": "timestamp",
    "used": false
}
```

Bucket: notification_queue

```
{
    "user_id": "ref → users._id",
    "email": "string",
    "tipo": "bloqueo | actividad_sospechosa | reset_password",
    "metadata": {
        "ip": "string",
        "dispositivo": "string",
        "user_agent": "string",
        "timestamp": "timestamp"
    },
    "intentos_envio": 0,
    "created_at": "timestamp"
```

}

Neo4j

Nodos

Label	Propiedades	Constraint / Índice
:User	userId (String), username (String), fullName (String), avatar (String)	UNIQUE userId; INDEX username; INDEX fullName
:Course	courseId (String), code (String), name (String), publicado (Boolean), estado (String)	UNIQUE courseId

Relaciones

Relación	Dirección	Propiedades	Descripción
ENROLLED_IN	(:User)→(:Course)	enrolledAt (DateTime), status	Matrícula de estudiante
TEACHES	(:User)→(:Course)	since (DateTime)	Docente del curso
FRIENDS_WITH	(:User)→(:User)	status (pending|active), since	Solicitud o amistad activa

Nota: al aceptar una amistad se crean dos relaciones FRIENDS_WITH (ambas direcciones) para facilitar consultas sin dirección sin costo adicional en runtime.

Cassandra

Tablas: login_attempts_by_user

```
CREATE TABLE login_attempts_by_user (
    user_id      uuid,
    timestamp    timestamp,
    ip           text,
    dispositivo  text,
    user_agent   text,
    exitoso      boolean,
    location     text,
    motivo_fallo text,    -- 'credenciales_invalidas' | 'cuenta_bloqueada'
| null si exitoso
    PRIMARY KEY (user_id, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Tablas: login_attempts_by_ip

```
CREATE TABLE login_attempts_by_ip (
    ip           text,
    timestamp    timestamp,
    user_id      uuid,
    dispositivo  text,
    user_agent   text,
    exitoso      boolean,
    location     text,
    PRIMARY KEY (ip, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Tablas: suspicious_activity_by_user

```
CREATE TABLE suspicious_activity_by_user (
    user_id      uuid,
    timestamp    timestamp,
    tipo         text,
    descripcion  text,
    ip           text,
```

```
    dispositivo  text,
    user_agent   text,
    notificado   boolean,
    resuelto     boolean,
    PRIMARY KEY (user_id, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Tablas: access_history_by_user

```
CREATE TABLE access_history_by_user (
    user_id      uuid,
    timestamp    timestamp,
    ip           text,
    dispositivo  text,
    user_agent   text,
    tipo_acceso  text,    -- 'login' | 'logout' | 'session_expired' |
'remember_me'
    resultado    text,    -- 'exitoso' | 'fallido' | 'bloqueado'
    location     text,
    PRIMARY KEY (user_id, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Tablas: audit_logs_by_user

```
CREATE TABLE audit_logs_by_user (
    user_id      uuid,
    timestamp    timestamp,
    accion       text,      -- ver valores abajo
    entidad      text,      -- 'course' | 'section' | 'evaluation' |
'enrollment' | 'user' | ...
    entidad_id   uuid,
    detalle      text,      -- JSON con campos cambiados (antes/después)
    ip           text,
    resultado    text,      -- 'ok' | 'error' | 'denegado'
    PRIMARY KEY (user_id, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Tablas: audit_logs_by_date

```
CREATE TABLE audit_logs_by_date (
    fecha        date,
    timestamp    timestamp,
    user_id      uuid,
    accion       text,
```

```
    entidad      text,
    entidad_id   uuid,
    detalle      text,
    ip           text,
    resultado    text,
    PRIMARY KEY (fecha, timestamp)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Distribución de Despliegue

Recursos en la Nube:

- Mongo DB Atlas

Recursos en Contenedores:

- Cassandra DB
- Neo4j DB
- Redis DB

Recursos en Server (LocalHost):

- Frontend: HTML+CSS+JS
- Backend: NodeJS+ExpressJS

Explicación de APIs con sus Historias de Usuario

Autenticación y Sesión:

POST /auth/register → Registro de usuario (HU-01)

POST /auth/login → Inicio de sesión (HU-02)

POST /session/logout → Cerrar sesión (HU-05)

POST /session/invalidate-all → Cerrar todas las sesiones activas (HU-05)

GET /session/validate → Validar sesión / cookie "Recordarme" (HU-06)

Gestión de Contraseñas:

POST /password/forgot → Solicitar enlace de recuperación (HU-07)

POST /password/reset → Restablecer contraseña con token (HU-07)

PATCH /password/change → Cambiar contraseña autenticado (HU-08)

Gestión de Cursos (Docentes):

POST /courses → Crear curso (HU-11)

POST /courses/:courseId/sections → Agregar sección/sub-sección (HU-12)

POST /courses/:courseId/sections/:sectionId/content → Agregar contenido (HU-13)

PATCH /courses/:courseId/publish → Publicar curso (HU-15)

GET /courses/:courseId/students → Estudiantes matriculados (HU-16)

GET /courses/teacher/:teacherId → Cursos del docente (HU-18)

POST /courses/:courseId/clone → Clonar curso (HU-19)

GET /courses/:courseId/sections → Obtener secciones

Matriculación y Contenido (Estudiantes):

GET /enrollment/search → Buscar cursos publicados (HU-20)

POST /enrollment → Matricularse (HU-21)

GET /enrollment/student/:studentId → Cursos matriculados (HU-22)

GET /enrollment/:courseId/content → Ver contenido (HU-23)

Evaluaciones:

POST /evaluations → Crear evaluación (HU-14)

POST /evaluations/:evalId/submit → Enviar evaluación (HU-24)

GET /evaluations/student/:studentId/course/:courseId → Resultados por curso (HU-25)

GET /evaluations/:evalId/results → Resultados globales (docente)

Mensajes y Comunicación:

POST /messages/course → Enviar consulta de curso (HU-26/HU-17)

GET /messages/course/:courseId/threads → Hilos del curso (HU-17)

GET /messages/thread/:threadId → Mensajes de hilo (HU-17)

POST /messages/direct → Mensaje directo (HU-30)

GET /messages/direct/:userId/:otherUserId → Conversación entre usuarios (HU-30)

GET /messages/conversations/:userId → Lista de conversaciones (HU-30)

Social y Amistades:

GET /social/users/search → Buscar usuarios (HU-29)

POST /social/friends/request → Enviar solicitud (HU-28)

PATCH /social/friends/request/:requesterId → Aceptar/rechazar (HU-28)

GET /social/friends/:userId → Lista de amigos (HU-28)

GET /social/friends/:userId/courses → Cursos de amigos (HU-28)

GET /social/courses/:courseId/mates → Compañeros de curso (HU-27)

Administración y Seguridad:

GET /admin/activity/:userId → Logs de actividad (HU-10)

GET /admin/security-events → Eventos de seguridad (HU-10/HU-04)

GET /admin/audit/:tableName → Audit trail por entidad

GET /admin/audit-by-date/:fecha → Audit trail por fecha

GET /admin/login-attempts/:userId → Intentos de login por usuario (HU-10)

GET /admin/login-attempts-ip/:ip → Intentos de login por IP (HU-10)

Salud del Sistema:

GET /health → Estado del sistema / base de datos

Uso de IA

Primero, es importante destacar que la IA no fue utilizada para generar el proyecto completo, sino como una guía complementaria. A pesar de ello, las herramientas de inteligencia artificial (IA), sí se utilizaron como apoyo en el proceso de desarrollo.

Generación de ideas iniciales

- Revisión de código
- Detección de posibles errores
- Sugerencias de mejora en estructura y organización

Herramientas de Inteligencia Artificial Utilizadas:

- ChatGPT(versión gratuita): IA Generativa
- Claude (versión gratuita): IA Generativa
- Windsurf (version gratuita): IDE con IA Generativa

Secciones uso de IA:

Arquitectura:

Las decisiones de arquitectura del sistema fueron definidas inicialmente por el equipo. Posteriormente, se consultó a herramientas como ChatGPT y Claude para:

- Validar la estructuración de las carpetas
- Sugerir estructuras comunes en proyectos tipo cliente-servidor de gestión educativa
- Proponer buenas prácticas en separación de responsabilidades y modularización

Las sugerencias obtenidas fueron analizadas y adaptadas, manteniendo control total sobre la arquitectura final implementada.

Frontend:

El desarrollo del frontend, se utilizó principalmente con la herramienta Claude para:

- Implementar una base sólida para la estructura (index.html) y estilos (styles.css) para la interfaz gráfica, de manera que fuera visualmente agradable y que ayudara a que fuera intuitiva cumpliendo con uno de los requerimientos no funcionales.
- Sugerir organización del código JavaScript
- Identificar posibles errores o redundancias
- Proponer mejoras en la interacción con la API

Se ajustó manualmente las llamadas a la API, y además se encargó de sugerir aspectos de depuración y en la corrección de los errores de integración

Backend:

En el backend, la IA se utilizó como herramienta de revisión y apoyo:

- Análisis de controllers y models
- Sugerencias de organización del código
- Identificación de posibles mejoras en manejo de errores
- Revisión de lógica en endpoints

Revisión docker compose:

La IA permitió pulir el docker compose. Se hizo una versión preliminar del docker compose para que contuviera todas las bases de datos y posteriormente se pidió a la IA analizar las características de este para ver si se asocia con la actualidad.

Conclusiones

El desarrollo de TecDigitalito permitió aplicar de forma práctica el uso de múltiples bases de datos especializadas en un mismo sistema, demostrando que no existe una solución única que se adapte óptimamente a todos los patrones de acceso y tipos de datos. La elección de MongoDB, Redis, Neo4j y Cassandra respondió a necesidades técnicas concretas: cada base de datos fue seleccionada según sus fortalezas y no por conveniencia general, lo cual representa una decisión de diseño más madura y fundamentada.

La separación de responsabilidades entre bases de datos resultó ser uno de los aprendizajes más valiosos del proyecto. Delegar la persistencia principal a MongoDB, la velocidad de acceso a datos efímeros a Redis, las relaciones sociales a Neo4j y la auditoría a Cassandra permitió que cada componente operara dentro de sus condiciones óptimas, evitando sobrecargar una sola base de datos con responsabilidades para las que no fue diseñada.

El uso del patrón MVC en el backend, junto con Express como capa de enrutamiento, facilitó mantener el código organizado y con responsabilidades bien delimitadas (ESIC, s. f.). Esto resultó especialmente importante dado el volumen de endpoints implementados y la necesidad de conectar simultáneamente con cuatro sistemas de almacenamiento distintos.

La incorporación de herramientas de inteligencia artificial como apoyo durante el desarrollo demostró ser una práctica útil cuando se utiliza de forma responsable. La IA agilizó tareas de revisión, detección de errores y generación de estructura base, pero las decisiones arquitectónicas y de diseño permanecieron siempre bajo el criterio del equipo, lo cual es fundamental para garantizar la comprensión real del sistema construido.

Finalmente, el proyecto evidenció que la planificación del modelo de datos debe anteceder al desarrollo del código. Decisiones como el diseño orientado a consultas en Cassandra, la desnormalización estratégica en Neo4j o el uso de índices compuestos en MongoDB tienen un impacto directo en el rendimiento y la escalabilidad del sistema, y son difíciles de corregir una vez que el desarrollo está avanzado.

Referencias Bibliográficas

Apache Cassandra (s. f.). | *Apache Cassandra Documentation*. Apache Cassandra. https://cassandra-apache-org.translate.goog/_/index.html?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Hebda, A. (2025, 14 febrero). Enviar Email en JavaScript: Tutorial con Código [2026]. *Mailtrap*. https://mailtrap.io/es/blog/javascript-send-email/

ESIC (s. f.). *Modelo-Vista-Controlador: qué es, cómo funciona y ventajas | ESIC*. https://www.esic.edu/rethink/tecnologia/modelo-vista-controlador-que-es-como-funciona-y-ventajas-c

MongoDB (s. f.). MongoDB Documentation. https://www.mongodb.com/es/docs/

Neo4j (s. f.). *Connection - Neo4J JavaScript Driver Manual*. Neo4j Graph Data Platform. https://neo4j.com/docs/javascript-manual/current/connect/

NodeJS (s. f.). *Introduction to Node.js | Node.js Learn*. https://nodejs.org/learn/getting-started/introduction-to-nodejs

Redis (2026, 25 febrero). *Getting Started with Node.js and Redis: node-redis & ioredis Guide*. Redis. https://redis.io/tutorials/develop/node/gettingstarted/

Schaefer, L. (2026, 4 marzo). Connect to a MongoDB Database Using Node.js. *MongoDB*. https://www.mongodb.com/blog/post/quick-start-nodejs-mongodb-how-to-get-connected-to-your-database